Library implementation (see chapter 5 for more details). On these platforms, a simple lock-based data structure might actually be more appropriate, but there's more to it than that; before choosing an implementation, you must identify your requirements and profile the various options that meet those requirements.

So, back to the beginning with the simplest of data structures: a stack.

7.2.1. Writing a thread-safe stack without locks

The basic premise of a stack is relatively simple: nodes are retrieved in the reverse order to which they were added—last in, first out (LIFO). It's therefore important to ensure that once a value is added to the stack, it can safely be retrieved immediately by another thread, and it's also important to ensure that only one thread returns a given value. The simplest stack is just a linked list; the `head` pointer identifies the first node (which will be the next to retrieve), and each node then points to the next node in turn.

Under such a scheme, adding a node is relatively simple:

1. Create a new node.
2. Set its `next` pointer to the current `head` node.
3. Set the `head` node to point to it.

This works fine in a single-threaded context, but if other threads are also modifying the stack, it's not enough. Crucially, if two threads are adding nodes, there's a race condition between steps 2 and 3: a second thread could modify the value of `head` between when your thread reads it in step 2 and you update it in step 3. This would then result in the changes made by that other thread being discarded or even worse consequences. Before we look at addressing this race condition, it's also important to note that once `head` has been updated to point to your new node, another thread could read that node. It's therefore vital that your new node is thoroughly prepared *before* `head` is set to point to it; you can't modify the node afterward.